MATH 566: Lecture 1 (08/25/2026)

This is Math 466/566 Network Optimization
I'm Bala Krishnamoorthy (Call me Bala).

Today
* syllabus, logistics
* The Königsberg bridges problem
* Network flow problems

A field that is closely related to network optimization is graph theory. While we will use several results that might also typically be presented in a graph theory course (Math 453/553), we will concentrate more on the "network" aspects.

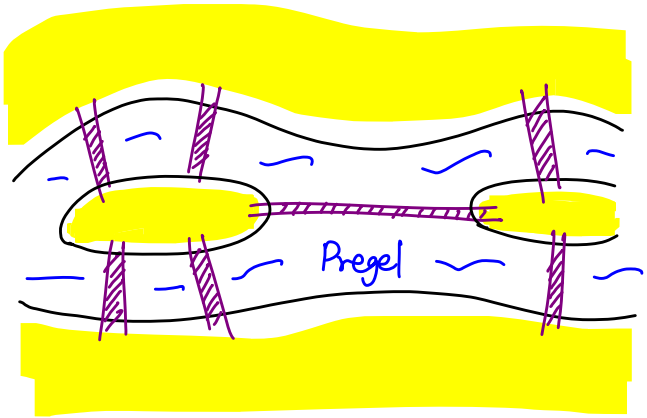
One key difference between "graphs" and "networks" as used conventionally, is that the arcs (or edges) in networks have capacities. In other words, one could think of them as pipes with limits on how much fluid could be sent through them at a time. On the other hand, edges in "graphs" are typically "lines" drawn between the points representing the nodes (or vertices). Thus, we will adopt a "flow" point of view.

Another difference between our approach and that of a typical graph theory class is that we will emphasize efficient algorithms to solve the optimization problems on networks. We will also explore numerous applications from various fields, including science, engineering, and business, of the different network problems we will study.

We will also emphasize **implementation of several algorithms**. Thus, there will be a sizeable programming component in this class.

The Königsberg Bridges Problem

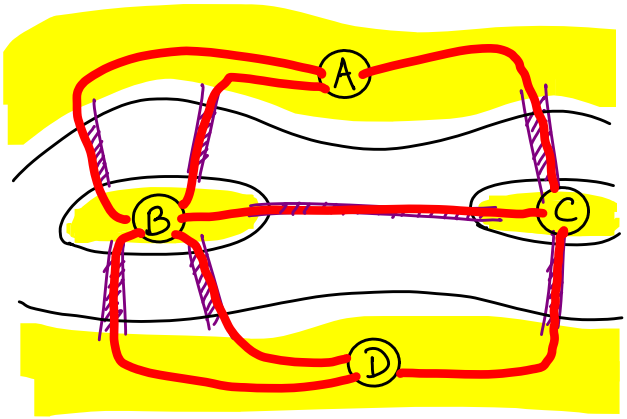
We start will a historical note. The first paper in graph theory (and, by extension, in network optimization) was published by Euler in 1736. He was visiting Königsberg in Prussia (now in Kaliningrad) The river Pregel flows through the town, and had seven bridges across it.

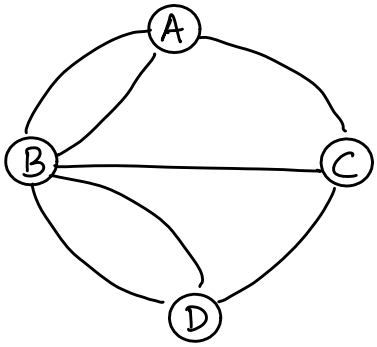


The citizens asked him if one could cross each of the seven bridges exactly once, and return to the starting point?

It was generally considered impossible to do. Euler characterized when it would be possible.

Here is a model for this problem. We represent each land mass by a node, and each bridge by an edge connecting the nodes representing the two land masses in question.





graph

We can restate the question as follows. In the graph, is it possible to start at A, say, traverse each edge exactly once, and return to A?

Since we return to where we started, such a "walk" is a cycle. Such a cycle, which traverses each edge exactly once, is called an Eulerian cycle. Hence we can ask "Does there exist an Eulerian cycle in the graph"?

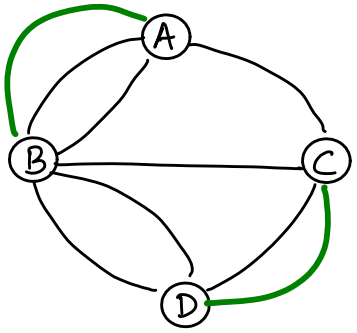
Theorem An undirected graph has an Eulerian cycle iff

* every node has an even degree, and
* the graph is connected, i.e., every pair of nodes has a "path" between them.

The degree of a node is the number of edges connected to it. We will not get into the details of this result right now. But one could get the intuition behind the even degree requirement. We should be able to "come in" to a node on an edge, and "go out" on another edge to a different node.

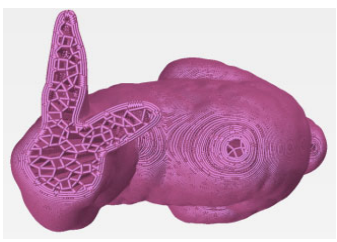
Many results (theorems) we present in this course will have a similar nature – intuitive to grasp and understand, and easy to state (may be not always easy to prove 😐!).

In the present case, if there were two more bridges as shown here, there would exist an Eulerian cycle.



Notice that all nodes have even degrees now (A,C,D:4, B:6).

An application of Eulerian cycles in 3D printing: check out https://arxiv.org/abs/1908.07452 !



A Related problem (Hamiltonian cycle problem): Does there exist a "cycle" that visits every node exactly once? This is the traveling salesman problem (TSP).

There are many applications of the TSP. It is one of the most well-studied network optimization problems. One typical application is in chip design, where the robotic arm printing the circuit pattern on the chip has to be programmed to finish printing the entire circuit in an optimal fashion in order to minimize the time spent.

We will now introduce several network optimization problems, which we will study in detail.

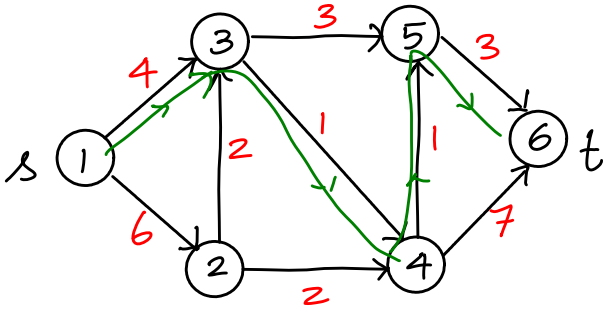
One more point about notation. In graph theory, it is common to call a graph as $G = (V, E)$, where V is the set of vertices and E the set of edges (undirected, by default). We will use the notation $G = (N, A)$ to denote a network, where N is the set of nodes and A is the set of arcs, which are directed by default.

Network Flow Problems

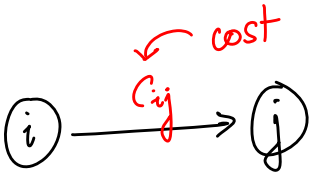
① Shortest path problem

Objective: find a path of minimum cost (length) from an origin (or source) node s to a destination (or sink) node t in a network with node set N and arc set A.

Here is an example:



Notation:



optimal or shortest path is shown in green

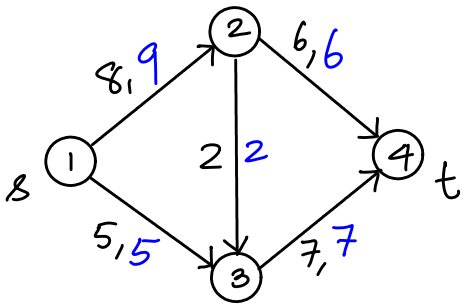
A typical application is driving directions on Google Maps. The costs c_{ij} could just be the distances between the cities modeled by the nodes, or could capture other relevant info such as traffic, tolls, etc.

Notice that the only parameter specified for the arcs is the cost, and the nodes themselves have no additional attributes (except for two of them being specified as s and t).

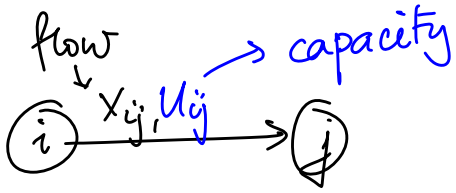
(2) Maximum flow problem (or max-flow)

Objective: Find a "feasible" flow (i.e., a solution that honors capacity limits on arcs) that sends maximum flow from a source node s to a sink node t.

Example:



notation:



max flow value $v = 13$.

Typical examples include traffic flow management – in road transportation networks or in computer networks.

Notice that we are working with directed arcs. This will be the default mode going forward, unless mentioned otherwise.

In the above instance, notice that the total flow coming into nodes 2 and 3 is equal to the total flow going out of that node. These nodes are "transshipment nodes", i.e., there is no loss or addition of material in these nodes. For the node s, we assume there is an infinite supply of material to be sent to node t, honoring the capacities.

We point out that the set of parameters specified in max flow include u_{ij}'s, the capacities on arcs. Notice there are no costs (c_{ij}) specified (unlike in the shortest path case).

③ Minimum cost flows (min-cost flow problem)

Objective: determine a least-cost shipment of commodity through a network in order to satisfy demands at certain nodes using supply at certain other nodes.

We will present an example of MCF in the next lecture...

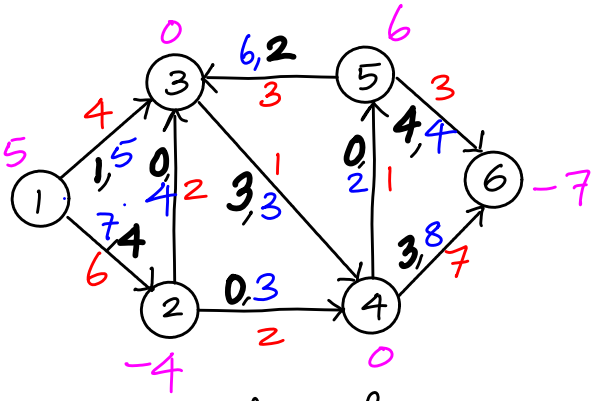
MATH 566: Lecture 2 (08/27/2026)

Today:
* Formal definition of MCF
  - SP, MF as cases of MCF
* Circulation, transportation, seat-sharing problem

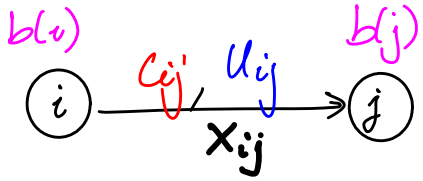
We first finish the introduction to min-cost flow (MCF) problem...

Recall: The objective of MCF is to determine a least cost shipment of a commodity through a network in order to satisfy demands at certain nodes using supply at certain other nodes.

Here is an illustration:



X_{ij}'s shown here form one feasible solution — not necessarily an optimal one.



$b(i)$: supply/demand

Nodes 1, 5: supply nodes (S)
2, 6: demand nodes (D)
3, 4: transshipment nodes.

We assume $\sum_{i \in N} b(i) = 0$ by default. In other words, the total supply in all supply nodes (S) is equal to the total demand in all demand nodes (D). The goal is to ship the supply from S nodes to D nodes at the minimum total cost while honoring capacity limits on arcs.

In min-cost flow, we specify costs (c_{ij}), capacities (u_{ij}), and supply/demand values at nodes (b_i).

We now specify the mathematical model for the MCF problem. Even though we introduce the model now, we will not solve the problems using this model – we will present efficient algorithms that exploit the network structure better (which the model does not do).

Notation

↗ G for "graph"

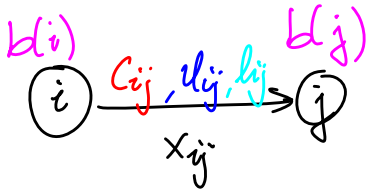
$G = (N, A)$; G: directed network, N: set of nodes, A: set of directed arcs.

$|N| = n$ (# nodes), $|A| = m$ (# arcs).

c_{ij} : unit cost on arc $(i,j) \in A$ (can be $>0, <0$, or $=0$)

u_{ij} : capacity of $(i,j) \in A$ (>0) → upper bound

l_{ij} : lower bound of $(i,j) \in A$ (≥ 0).



By default, $l_{ij} = 0$, unless mentioned otherwise.

$b(i)$: supply/demand at node $i \in N$

$> 0 \Rightarrow$ i is a supply node

$< 0 \Rightarrow$ i is a demand node ↗ demand of $-b(i)$ at node i

$= 0 \Rightarrow$ i is a transshipment node

$b(i), c_{ij}, l_{ij}, u_{ij}$: data (known with certainty). These parameters do not depend on x_{ij}'s, which are variables.

Goal: Find flow x_{ij} $\forall (i,j) \in A$ such that bounds are satisfied, supply/demand is "met" at each node $i \in N$, and overall cost is minimum.

We assume $\sum_{i \in N} b(i) = 0$ by default, i.e., total supply = total demand.

Here is the optimization model for the min-cost flow problem:

$$\min \left\{ \sum_{(i,j) \in A} c_{ij} x_{ij} \right\} \quad \text{(1)}$$

(minimize; total cost)

subject to (s.t.)

$$\sum_{(i,j) \in A} x_{ij} - \sum_{(j,i) \in A} x_{ji} = b(i) \quad \forall\, i \in N \quad \text{(2)}$$

(for all)

outflow − inflow = supply/demand

$$l_{ij} \le x_{ij} \le u_{ij} \quad \forall\, (i,j) \in A \quad \text{(3)}$$

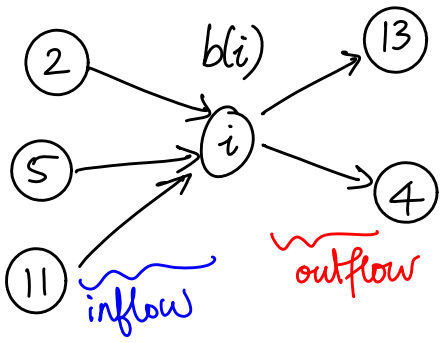
Mathematical notation: $\in$: "element of", $\forall$: "for all", $\sum$: sum.

In words, we want to minimize the total cost (1), subject to meeting the supply/demand constraints at each node (2), and the bounds on flow in each arc (3).

This is a linear optimization problem (or a linear program, LP). But we won't solve these instances the same way we solve LPs. These problems are simpler than the general LPs — the underlying network structure allows one to design efficient algorithms, which run faster than default algos to solve LPs!

(1) models the total cost incurred. On arc (i,j), one unit of flow incurs a cost of c_{ij}. Hence x_{ij} units incur $c_{ij}x_{ij}$. Summing up all such cost terms (over all arcs in the arc set A) gives the total cost. The goal is to find a flow, i.e., all x_{ij} values, that minimizes the total cost.

The flow must satisfy constraints (2) and (3). (2) are the flow balance (or mass balance) constaints, which you could think of as "conservation of flow (or mass)."



$x_{i,13} + x_{i,4}$ } outflow

$x_{2i} + x_{5i} + x_{11,i}$ } inflow

$b(i)$: supply/demand

outflow − inflow = supply/demand

There is no flow "lost" or "appearing out of the blue". In words, "what comes in + what is produced or used = what goes out".

The flow must honor the lower and upper bounds on each arc, as specified by constraints (3). The default value for all l_{ij} is zero. If u_{ij} is not specified, it is taken as $+\infty$ (some large number in practice).

We also assume here that $\sum_{i \in N} b(i) = 0$, i.e., total supply is equal to total demand. There are generalizations where this condition might not hold, when (2) may not be written as '=' for all i.

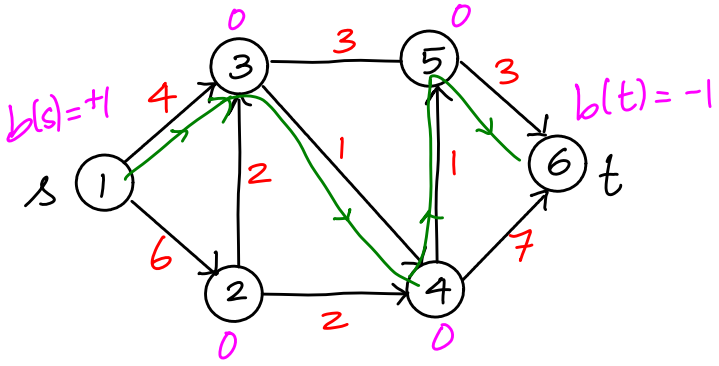
We now show that the shortest path and the max flow problems are special cases of the min cost flow problem.

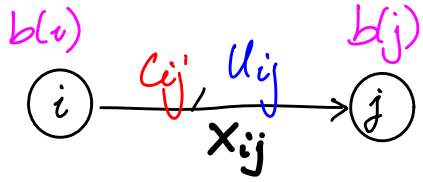
(*) When "formulating" or "modeling" a problem as a network flow problem, you need to specify the network $G=(N,A)$, i.e., what is the node set, the arc set, and what are the associated parameters $(b(i), l_{ij}, u_{ij}, c_{ij})$. (M some big >0 number in practice)

Default values: $b(i)=0$, $l_{ij}=0$, $c_{ij}=0$, $u_{ij}=+\infty$.

Shortest Path Problem as a min-cost flow Problem

Consider the example for shortest path from Lecture 1.



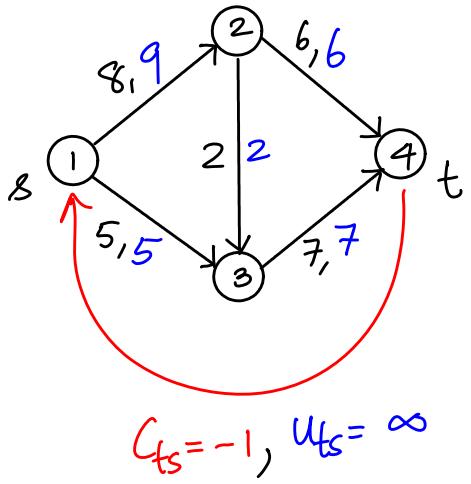


Use same network $G=(N,A)$ as in the shortest path problem. We set $b(s)=+1$, $b(t)=-1$, $b(i)=0$ $\forall i \neq s,t$, $i \in N$. Then we set $l_{ij}=0$, $u_{ij} \geq 1$ $\forall (i,j) \in A$.

Solving the MCF instance here is equivalent to sending one unit of flow from s to t at the least cost. Think of sending one car from the origin to destination. To detail the argument, we can show $x_{ij}=1$ or $x_{ij}=0$ here (there are some deeper assumptions critical here). Then argue that the $(i,j) \in A$ with $x_{ij}=1$ precisely gives the shortest path.

Max Flow as special case of MCF

In the previous instance (shortest path), we did not have to add any extra nodes or arcs. To model the max flow problem as a min cost flow problem, we add a single extra arc.



Add (t,s) to A, with $c_{ts}=-1$, $u_{ts}=+\infty$.

Set $l_{ij}=0 \quad \forall (i,j)\in A$ (including l_{ts})

$c_{ij}=0 \quad \forall (i,j)\in A \neq (t,s)$.

$b(i)=0 \quad \forall i \in N$.

Since $c_{ts}=-1$ (we could set it to any value <0, while keeping all other $c_{ij}=0$), min cost flow will try to send as much flow as possible on arc (t,s). And all flow on (t,s) enters node s. Further, this flow is also equal to the net flow out of node t. In other words, the flow on (t,s) is indeed the maximum flow from s to t. Since $u_{ts}=\infty$, the value of the max flow is determined by the u_{ij} values of the original network, as it should be.

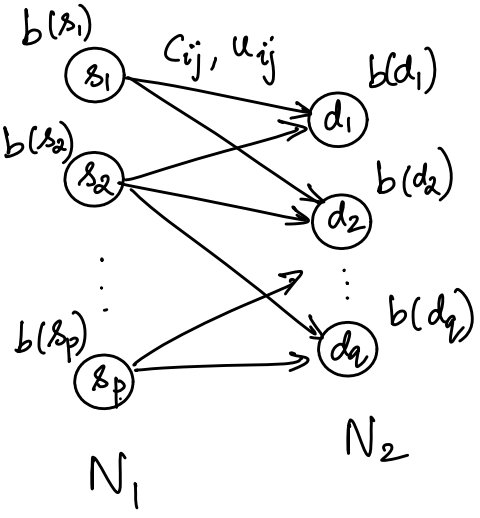
Notice that the flow just circulates around the network here. There is no flow coming in or going out at any of the nodes. This is an instance of the **circulation problem**, which is the min cost flow problem with $b(i)=0$ for all nodes i. This is the fourth network flow problem class (after shortest path, max flow and min cost flow).

→ min cost circulation, to be exact

A typical example of the circulation problem is the scheduling of airplanes operated by a firm, e.g., Alaska Airlines. Each city/destination served is a node, and the total number of planes remains same within the network. If a service is required between, say, Seattle and Spokane, the l_{ij} for that arc is set to 1, ensuring one plane flies from Seattle to Spokane.

(5) Transportation problem

This is a special case of mincost flow where the node set $N = N_1 \cup N_2$, ("union") with N_1 being supply nodes and N_2 being demand nodes. Hence, $b(i) > 0 \ \forall i \in N_1$, and $b(j) < 0 \ \forall j \in N_2$. Further, all arcs $(i,j) \in A$ have $i \in N_1$ and $j \in N_2$. The arcs have c_{ij}, l_{ij}, and u_{ij} specified.



Notice that there are no arcs within N_1 or within N_2.

We assume that $\sum_{i \in N_1} b(i) = -\sum_{j \in N_2} b(j)$,

i.e., total supply = total demand.

Such an instance is a **balanced transportation problem**. In the unbalanced case, there might be penalties for unmet demand or for unused supply.

There are no transshipment nodes in the standard transportation problem. Indeed, if there were a node with zero supply within N_1, we could remove it without changing the main problem. Similar is the case with a zero demand node in N_2.

The default example is that of shipping of a commodity between warehouses and retailers, with the former set forming the supply nodes, and the retailers forming the demand nodes. If there is an option to ship the commodity from warehouse i to retailer j, arc (i,j) is included in the network. For instance, if there is a truck available to ship items from the Seattle warehouse to the Vancouver store; the capacity and cost associated with the truck are modeled as u_{ij} (and l_{ij}, if there is a minimum), and c_{ij}.

Read section 1.3 on modeling applications as various network flow problems. We will discuss one such problem — the seat sharing problem — in detail in the next lecture. A handout on this problem is posted on the course web page.

Seat Sharing Problem

(AMO 1.8, page 21) Several families are planning a shared car trip on scenic drives in the Cascades in Washington. To minimize the possibility of any quarrels, the organizers of the trip want to assign individuals to cars so that **no two members of a family are in the same car**. Formulate this problem as a network flow problem.

More in the next class...